File: .env

```
POSTGRES_USER=meterstack_user
POSTGRES_PASSWORD=your_password
POSTGRES_DB=meterstack_db
DJANGO_SECRET_KEY=your-secret-key
DJANGO_DEBUG=False
DJANGO_ALLOWED_HOSTS=localhost,127.0.0.1
```

File: .env.example

```
# Django
DJANGO_SECRET_KEY=change-me-to-a-random-secret-key
DJANGO_DEBUG=False
DJANGO_ALLOWED_HOSTS=localhost,127.0.0.1

# PostgreSQL
POSTGRES_DB=meterstack_db
POSTGRES_USER=meterstack_user
POSTGRES_PASSWORD=change_me
POSTGRES_HOST=db
POSTGRES_PORT=5432
```

File: .gitignore.py

File: .pre-commit-config.yaml.py

```
repos:
  - repo: https://github.com/psf/black
    rev: 23.12.1
    hooks:
      - id: black
        language_version: python3.12
  - repo: https://github.com/pycqa/flake8
    rev: 7.0.0
    hooks:
      - id: flake8
        args: ['--max-line-length', '120', '--extend-ignore', 'E203,W503']
  - repo: local
    hooks:
      - id: django-tests
        name: Run Django tests
        entry: python manage.py test --noinput
        language: system
        pass_filenames: false
        types: [python]
```

File: README.md

```
# MeterStack ? Multi?Tenant API Usage Billing System

**A production?ready, multi?tenant, metered API platform** built with Django, Django REST Framework, and PostgreSQL.
Designed to sell data APIs, meter every call, bill tenants automatically, and provide powerful dashboards.

---

## Table of Contents

- [Overview](#overview)
- [Features](#features)
- [Architecture & Design](#architecture--design)
- [Professional Practices](#professional-practices)
- [Setup (with Docker)](#setup-with-docker)
- [API Documentation](#api-documentation)
- [Running Tests](#running-tests)
- [Concurrency & Race Condition Handling](#concurrency--race-condition-handling)
- [Project Structure](#project-structure)
- [Roadmap](#roadmap)
- [License](#license)

---

## Overview

MeterStack allows platform owners to:

- Register **tenants** (companies) and their admin users.
- Define **endpoints** (API products) with custom pricing (per call + per KB).
- Provide a public data endpoint that tenants' users can call.
- **Automatically record every call** in a thread?safe, concurrency?safe manner.
- Generate **monthly invoices** with line items, applying tenant?specific pricing plans and discounts.
- Give tenant admins a **real?time usage dashboard** with filtering, search, and cursor pagination.
- Provide **reporting and analytics** with caching and period?over?period comparisons.
- Support **budget alerts** to warn tenants when spending exceeds a threshold.

The entire system is containerised with Docker and ready for production deployment.

---

## Features

### ? Multi?Tenant Authentication & Permissions
- Custom user model with roles (`admin`, `member`).
- JWT authentication with custom claims (`tenant_slug`, `role`).
- Object?level permissions guaranteeing complete tenant data isolation.

### ? Public Data Endpoint (Metered API)
- `POST /api/v1/data/` ? tenants' users call this to get (mock) data.
- Every successful request automatically records usage and updates monthly summaries.
- Concurrency?safe recording using `select_for_update()` and atomic transactions.

### ? Usage Tracking & Summaries
- Detailed `UsageRecord` table for every call.
- Pre?aggregated `UsageSummary` table per tenant, endpoint, and month for instant billing queries.
- Retry decorator for deadlock resilience.

### ? Billing Engine
- `PricingPlan` model with base fee, per?call and per?KB rates, and discounts.
- `BillingService.generate_invoice()` that aggregates monthly summaries, applies pricing plans, and creates an `Invoic
- Management command (`generate_invoices`) to bill all active tenants for any given month.
- Unique constraints prevent duplicate invoices.

### ? Admin Dashboard & Reporting
- `GET /api/v1/usage/` ? paginated, filterable, searchable list of usage records (tenant admins only).
```

- `GET /api/v1/dashboard/summary/` ? top endpoints + current?vs?previous month comparison.
- `GET /api/v1/dashboard/trends/` ? daily usage trends.
- Caching layer (cache?aside pattern) for expensive reports.
- Custom model managers for reusable analytics queries.

?? Security & Robustness
- Database constraints: unique invoices, non?negative totals, duplicate prevention.
- Custom exception handler returning consistent JSON errors.
- Health?check endpoint (`/api/health/`) for monitoring.
- Production?grade logging (JSON format) with request?ID tracing.

? Extensive Test Suite
- Authentication and tenant isolation tests.
- Concurrency tests using `ThreadPoolExecutor` to prove zero lost or double?counted usage.
- Billing tests verifying invoice generation, duplicate prevention, and alerting.

? Swagger Documentation
- Interactive API docs at `/api/docs/` (Swagger UI).
- All endpoints annotated with `@extend_schema`.

Architecture & Design

- **Layered architecture** ? Models, Services, Views, Serializers, Permissions are separated.
- **Service layer** ? All business logic lives in dedicated service classes (`UsageService`, `BillingService`, `Report
- **Custom managers** ? Complex ORM queries encapsulated in model managers.
- **Cache?aside pattern** ? Dashboard reports cached for 5 minutes, invalidated on new usage.
- **Concurrency safety** ? `select_for_update()`, `F()` expressions, atomic transactions, and a deadlock?retry decorat

---

Professional Practices

Practice	How it's applied
Type hints	Throughout services, views, managers, and permissions.
Logging	Structured JSON logs with request IDs; INFO/WARNING/ERROR levels.
Context managers	`managed_cursor()` and custom timer context manager.
Clean architecture	Views only handle HTTP; services contain all business logic.
Multi?tenancy isolation	Tenant filtering at the database level + object?level permissions.
Caching	Low?level Django cache API with cache hits/misses logging.
Pre?commit hooks	`black`, `flake8`, and test runner enforced before every commit.
Dockerised	Multi?stage Dockerfile, docker?compose with PostgreSQL and Redis.
Swagger	Automatic API documentation with `drf?spectacular`.
Testing	Unit tests, integration tests, and concurrency stress tests.

Setup (with Docker)

1. **Clone the repository**

   ```bash
   git clone https://github.com/yourusername/meterstack.git
   cd meterstack/meterstack2
   ```

File: __init__.py

File: docker-compose.yml

```
services:
  # ---------------- Django ----------------
  web:
    build: .
    container_name: meterstack_web

    command: >
      sh -c "
      python manage.py migrate &&
      gunicorn --bind 0.0.0.0:8000 --workers 3 meterstack2.wsgi:application
      "

    volumes:
      - .:/app

    ports:
      - "8000:8000"

    env_file:
      - .env

    depends_on:
      db:
        condition: service_healthy

      redis:
        condition: service_started

  # ---------------- PostgreSQL ----------------
  db:
    image: postgres:15-alpine
    container_name: meterstack_db

    env_file:
      - .env

    volumes:
      - postgres_data:/var/lib/postgresql/data

    healthcheck:
      test:
        [
          "CMD-SHELL",
          "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"
        ]
      interval: 10s
      timeout: 5s
      retries: 5

  # ---------------- Redis ----------------
  redis:
    image: redis:7-alpine
    container_name: meterstack_redis

    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
      timeout: 5s
      retries: 5

volumes:
  postgres_data:
```

File: generate_pdf.py

```
import os
from fpdf import FPDF

# ---------- configuration ----------
PROJECT_ROOT = os.getcwd()
OUTPUT_PDF = os.path.join(PROJECT_ROOT, "meterstack_full_code.pdf")
EXCLUDE_DIRS = {
    '.git', '__pycache__', 'venv', 'env', '.idea', 'migrations', 'static',
    'node_modules', 'tests/__pycache__', 'billing/migrations/__pycache__',
    'usage/migrations/__pycache__', 'users/migrations/__pycache__',
    'tenants/migrations/__pycache__', 'meterstack2/__pycache__',
}
INCLUDE_EXTENSIONS = {
    '.py', '.txt', '.md', '.yml', '.yaml', '.sh', '.cfg', '.ini',
    '.env', '.env.example', '.sql', '.json', '.html', '.css', '.js',
    'Dockerfile', 'docker-compose.yml'
}
# -----------------------------------

def safe_encode(text):
    """Replace characters that can't be encoded in latin-1 with '?'."""
    return text.encode('latin-1', errors='replace').decode('latin-1')

pdf = FPDF()
pdf.set_auto_page_break(auto=True, margin=15)
pdf.add_page()
pdf.set_font("Courier", size=8)

for root, dirs, files in os.walk(PROJECT_ROOT):
    dirs[:] = [d for d in dirs if d not in EXCLUDE_DIRS and not d.startswith('.')]

    for file in sorted(files):
        ext = os.path.splitext(file)[1].lower()
        name = file.lower()
        if ext not in INCLUDE_EXTENSIONS and name not in INCLUDE_EXTENSIONS:
            continue

        filepath = os.path.join(root, file)
        relpath = os.path.relpath(filepath, PROJECT_ROOT)

        pdf.set_font("Courier", 'B', 10)
        pdf.cell(0, 10, safe_encode(f"File: {relpath}"), ln=True)
        pdf.set_font("Courier", size=8)

        try:
            with open(filepath, 'r', encoding='utf-8', errors='replace') as f:
                for line in f:
                    cleaned = line.rstrip('\n').expandtabs(4)
                    # replace tabs with spaces and ensure latin-1 compatibility
                    pdf.cell(0, 4, safe_encode(cleaned), ln=True)
        except Exception as e:
            pdf.cell(0, 4, safe_encode(f"Error reading file: {e}"), ln=True)

        pdf.add_page()

pdf.output(OUTPUT_PDF)
print(f"PDF generated: {OUTPUT_PDF}")
```

File: manage.py

```
#!/usr/bin/env python
"""Django's command-line utility for administrative tasks."""
import os
import sys


def main():
    """Run administrative tasks."""
    os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'meterstack2.settings')
    try:
        from django.core.management import execute_from_command_line
    except ImportError as exc:
        raise ImportError(
            "Couldn't import Django. Are you sure it's installed and "
            "available on your PYTHONPATH environment variable? Did you "
            "forget to activate a virtual environment?"
        ) from exc
    execute_from_command_line(sys.argv)


if __name__ == '__main__':
    main()
```

File: requirements.txt

```
django
djangorestframework
djangorestframework-simplejwt
django-filter
drf-spectacular
psycopg2-binary
python-dateutil
python-json-logger
gunicorn
```

File: urls.py

```
from django.contrib import admin
from django.urls import path, include
from drf_spectacular.views import SpectacularAPIView, SpectacularSwaggerView
from core.views import HealthCheckView

urlpatterns = [
    path('admin/', admin.site.urls),

    # OpenAPI schema & docs
    path('api/schema/', SpectacularAPIView.as_view(), name='schema'),
    path('api/docs/', SpectacularSwaggerView.as_view(url_name='schema'), name='swagger-ui'),

    # API v1
    path('api/v1/', include('users.urls')),
    path('api/v1/', include('tenants.urls')),
    path('api/v1/', include('usage.urls')),
    path('api/v1/', include('billing.urls')),
    path('api/health/', HealthCheckView.as_view(), name='health-check'),
]
```

File: billing__init__.py

File: billing\admin.py

```
from django.contrib import admin

from .models import (
    PricingPlan,
    Invoice,
    InvoiceLine,
    UsageAlert,
)


class InvoiceLineInline(admin.TabularInline):
    model = InvoiceLine
    extra = 0
    readonly_fields = ('endpoint', 'calls', 'amount')
    can_delete = False
    show_change_link = True


@admin.register(PricingPlan)
class PricingPlanAdmin(admin.ModelAdmin):
    list_display = (
        'tenant',
        'flat_monthly_fee',
        'discount_percent',
        'included_requests',
        'created_at',
    )
    search_fields = ('tenant__name',)
    raw_id_fields = ('tenant',)


@admin.register(Invoice)
class InvoiceAdmin(admin.ModelAdmin):
    list_display = (
        'tenant',
        'period',
        'subtotal',
        'discount',
        'total',
        'created_at',
    )
    list_filter = ('period', 'tenant')
    search_fields = ('tenant__name',)
    ordering = ('-period',)
    date_hierarchy = 'period'
    raw_id_fields = ('tenant',)
    inlines = [InvoiceLineInline]


@admin.register(InvoiceLine)
class InvoiceLineAdmin(admin.ModelAdmin):
    list_display = ('invoice', 'endpoint', 'calls', 'amount')
    search_fields = ('invoice__tenant__name', 'endpoint__name')
    raw_id_fields = ('invoice', 'endpoint')


@admin.register(UsageAlert)
class UsageAlertAdmin(admin.ModelAdmin):
    list_display = (
        'tenant',
        'monthly_budget',
        'threshold_percent',
        'is_enabled',
        'created_at',
    )
```

```
list_filter = ('is_enabled',)
search_fields = ('tenant__name',)
raw_id_fields = ('tenant',)
```

File: billing\apps.py

```
from django.apps import AppConfig


class BillingConfig(AppConfig):
    name = 'billing'
```

File: billing\filters.py

```
import django_filters

from .models import Invoice


class InvoiceFilter(django_filters.FilterSet):
    start_date = django_filters.DateFilter(
        field_name="period",
        lookup_expr="gte",
    )

    end_date = django_filters.DateFilter(
        field_name="period",
        lookup_expr="lte",
    )

    class Meta:
        model = Invoice
        fields = [
            "start_date",
            "end_date",
        ]
```

File: billing\models.py

```
from decimal import Decimal
from django.core.validators import MinValueValidator, MaxValueValidator
from django.db import models
from tenants.models import Tenant
from usage.models import Endpoint


class PricingPlan(models.Model):
    tenant = models.OneToOneField(
        Tenant,
        on_delete=models.CASCADE,
        related_name="pricing_plan",
    )
    flat_monthly_fee = models.DecimalField(
        max_digits=10,
        decimal_places=2,
        default=Decimal("0.00"),
        validators=[MinValueValidator(Decimal("0.00"))],
    )
    included_requests = models.PositiveIntegerField(default=0)
    discount_percent = models.DecimalField(
        max_digits=5,
        decimal_places=2,
        default=Decimal("0.00"),
        validators=[MinValueValidator(Decimal("0.00"))],
    )
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"{self.tenant.slug} Pricing Plan"


class Invoice(models.Model):
    tenant = models.ForeignKey(
        Tenant,
        on_delete=models.CASCADE,
        related_name="invoices",
    )
    period = models.DateField()
    subtotal = models.DecimalField(
        max_digits=12,
        decimal_places=2,
        default=Decimal("0.00"),
        validators=[MinValueValidator(Decimal("0.00"))],
    )
    discount = models.DecimalField(
        max_digits=12,
        decimal_places=2,
        default=Decimal("0.00"),
        validators=[MinValueValidator(Decimal("0.00"))],
    )
    total = models.DecimalField(
        max_digits=12,
        decimal_places=2,
        default=Decimal("0.00"),
        validators=[MinValueValidator(Decimal("0.00"))],
    )
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        constraints = [
            models.UniqueConstraint(
                fields=["tenant", "period"],
                name="unique_invoice_per_tenant_period",
            ),
```

```
            models.CheckConstraint(
                condition=models.Q(total__gte=0),
                name="invoice_total_non_negative",
            ),
            # New: ensure period is always the first day of a month
            models.CheckConstraint(
                condition=models.Q(period__day=1),
                name="invoice_period_must_be_first_of_month",
            ),
        ]
        indexes = [
            models.Index(fields=["tenant", "period"]),
        ]
        ordering = ["-period"]

    def __str__(self):
        return f"{self.tenant.slug} - {self.period}"


class InvoiceLine(models.Model):
    invoice = models.ForeignKey(
        Invoice,
        on_delete=models.CASCADE,
        related_name="lines",
    )
    endpoint = models.ForeignKey(
        Endpoint,
        on_delete=models.PROTECT,
    )
    calls = models.PositiveIntegerField()
    amount = models.DecimalField(
        max_digits=12, decimal_places=2,
        validators=[MinValueValidator(Decimal("0.00"))],
    )

    def __str__(self):
        return f"{self.invoice} - {self.endpoint.name}"


class UsageAlert(models.Model):
    tenant = models.OneToOneField(
        Tenant,
        on_delete=models.CASCADE,
        related_name="usage_alert",
    )
    monthly_budget = models.DecimalField(
        max_digits=12, decimal_places=2,
        validators=[MinValueValidator(Decimal("0.00"))],
    )
    threshold_percent = models.PositiveIntegerField(
        default=80,
        validators=[MinValueValidator(1), MaxValueValidator(100)],
    )
    is_enabled = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"{self.tenant.slug} Budget Alert"
```

File: billing\serializers.py

```
from rest_framework import serializers

from .models import Invoice


class InvoiceSerializer(serializers.ModelSerializer):
    class Meta:
        model = Invoice
        fields = [
            "id",
            "period",
            "subtotal",
            "discount",
            "total",
            "created_at",
        ]
```

File: billing\services.py

```
from __future__ import annotations

import logging
from datetime import date
from decimal import Decimal

from django.db import transaction

from tenants.models import Tenant
from usage.models import UsageSummary
from .models import Invoice, InvoiceLine, PricingPlan, UsageAlert

logger = logging.getLogger(__name__)


class BillingService:
    @staticmethod
    @transaction.atomic
    def generate_invoice(*, tenant: Tenant, period: date) -> Invoice:

        period = period.replace(day=1)

        summaries = (
            UsageSummary.objects
            .select_for_update()
            .filter(tenant=tenant, period=period)
        )

        if Invoice.objects.filter(tenant=tenant, period=period).exists():
            raise ValueError("Invoice already exists.")

        pricing = PricingPlan.objects.get(tenant=tenant)
        usage_cost = sum((summary.total_cost for summary in summaries), Decimal("0.00"))
        subtotal = pricing.flat_monthly_fee + usage_cost
        discount = subtotal * pricing.discount_percent / Decimal("100")
        total = subtotal - discount

        invoice = Invoice.objects.create(
            tenant=tenant,
            period=period,
            subtotal=subtotal,
            discount=discount,
            total=total,
        )


        lines = [
            InvoiceLine(
                invoice=invoice,
                endpoint=summary.endpoint,
                calls=summary.total_calls,
                amount=summary.total_cost,
            )
            for summary in summaries
        ]
        InvoiceLine.objects.bulk_create(lines)

        return invoice

    @staticmethod
    def check_budget_alert(*, invoice: Invoice) -> None:
        try:
            alert = UsageAlert.objects.get(
                tenant=invoice.tenant, is_enabled=True
            )
```

```
except UsageAlert.DoesNotExist:
    return

threshold_amount = (
    alert.monthly_budget
    * Decimal(alert.threshold_percent)
    / Decimal("100")
)
if invoice.total >= threshold_amount:
    logger.warning(
        "Budget alert | tenant=%s invoice_total=%s budget=%s threshold=%s%%",
        invoice.tenant.slug,
        invoice.total,
        alert.monthly_budget,
        alert.threshold_percent,
    )
```

File: billing\tests.py

```
from django.test import TestCase
from datetime import date
from decimal import Decimal
from django.test import TestCase
from billing.models import PricingPlan, Invoice
from .services import BillingService
from tenants.models import Tenant
from usage.models import Endpoint, UsageSummary
from datetime import date
from decimal import Decimal
from django.test import TestCase
from billing.models import UsageAlert
from billing.services import BillingService
from datetime import date
from django.core.management import call_command
from django.test import TestCase
from .models import Invoice


class BillingServiceTests(TestCase):

    def setUp(self):
        self.tenant = Tenant.objects.create(name="Acme")

        self.endpoint = Endpoint.objects.create(
            tenant=self.tenant,
            name="Weather API",
            path="/weather/",
            price_per_call=Decimal("0.05"),
        )

        PricingPlan.objects.create(
            tenant=self.tenant,
            flat_monthly_fee=Decimal("10.00"),
            discount_percent=Decimal("10.00"),
        )

        UsageSummary.objects.create(
            tenant=self.tenant,
            endpoint=self.endpoint,
            period=date(2026, 3, 1),
            total_calls=100,
            total_cost=Decimal("50.00"),
        )

    def test_generate_invoice(self):

        invoice = BillingService.generate_invoice(
            tenant=self.tenant,
            period=date(2026, 3, 1),
        )

        self.assertEqual(invoice.subtotal, Decimal("60.00"))
        self.assertEqual(invoice.discount, Decimal("6.00"))
        self.assertEqual(invoice.total, Decimal("54.00"))

        self.assertEqual(Invoice.objects.count(), 1)




class BillingCommandTests(TestCase):

    def setUp(self):
        self.tenant = Tenant.objects.create(name="Acme")
```

```
        self.endpoint = Endpoint.objects.create(
            tenant=self.tenant,
            name="Weather API",
            path="/weather/",
            price_per_call=Decimal("0.05"),
        )
        PricingPlan.objects.create(
            tenant=self.tenant,
            flat_monthly_fee=Decimal("10.00"),
            discount_percent=Decimal("10.00"),
        )
        UsageSummary.objects.create(
            tenant=self.tenant,
            endpoint=self.endpoint,
            period=date(2026, 3, 1),
            total_calls=100,
            total_cost=Decimal("50.00"),
        )

    def test_command_does_not_create_duplicate_invoice(self):
        call_command("generate_invoices", period="2026-03-01")
        call_command("generate_invoices", period="2026-03-01")

        self.assertEqual(Invoice.objects.count(), 1)



class BudgetAlertTests(TestCase):

    def test_budget_alert_is_triggered(self):

        UsageAlert.objects.create(
            tenant=self.tenant,
            monthly_budget=Decimal("100.00"),
            threshold_percent=80,
        )

        invoice = BillingService.generate_invoice(
            tenant=self.tenant,
            period=date(2026, 3, 1),
        )

        with self.assertLogs("billing.services") as logs:

            BillingService.check_budget_alert(
                invoice=invoice,
            )

        self.assertIn(
            "Budget alert",
            logs.output[0],
        )


# Create your tests here.
```

File: billing\urls.py

```
from django.urls import path

from .views import InvoiceListView

urlpatterns = [
    path(
        "invoices/",
        InvoiceListView.as_view(),
        name="invoice-list",
    ),
]
```

File: billing\views.py

```
from django_filters.rest_framework import DjangoFilterBackend

from rest_framework.filters import OrderingFilter
from rest_framework.generics import ListAPIView
from rest_framework.permissions import IsAuthenticated

from users.permissions import IsTenantAdmin

from .filters import InvoiceFilter
from .models import Invoice
from .serializers import InvoiceSerializer


class InvoiceListView(ListAPIView):
    serializer_class = InvoiceSerializer
    permission_classes = [
        IsAuthenticated,
        IsTenantAdmin,
    ]

    filter_backends = [
        DjangoFilterBackend,
        OrderingFilter,
    ]

    filterset_class = InvoiceFilter

    ordering_fields = [
        "period",
        "total",
    ]

    ordering = [
        "-period",
    ]

    def get_queryset(self):
        return (
            Invoice.objects
            .filter(tenant=self.request.user.tenant)
            .prefetch_related('lines')
            .order_by("-period")
        )
```

File: billing\management__init__.py

File: billing\management\commands__init__.py

File: billing\management\commands\generate_invoices.py

```
import logging
from datetime import date
from django.core.management.base import BaseCommand
from tenants.models import Tenant
from billing.services import BillingService
from billing.models import PricingPlan, Invoice, InvoiceLine, UsageAlert

logger = logging.getLogger(__name__)


class Command(BaseCommand):
    help = "Generate monthly invoices for all active tenants."

    def add_arguments(self, parser):
        parser.add_argument(
            "--period",
            type=str,
            help="Billing period (YYYY-MM-01). Example: 2026-03-01",
        )

    def handle(self, *args, **options):

        period = (
            date.fromisoformat(options["period"])
            if options["period"]
            else date.today().replace(day=1)
        )

        tenants = Tenant.objects.filter(is_active=True)



        for tenant in tenants:
            try:
                invoice=BillingService.generate_invoice(
                    tenant=tenant,
                    period=period,
                )
                BillingService.check_budget_alert(
                    invoice=invoice,
                )


                logger.info(
                    "Invoice generated for tenant '%s'",
                    tenant.slug,
                )

                self.stdout.write(
                    self.style.SUCCESS(
                        f"? {tenant.slug}"
                    )
                )

            except Exception as exc:

                logger.exception(
                    "Billing failed for tenant '%s'",
                    tenant.slug,
                )

                self.stdout.write(
                    self.style.ERROR(
                        f"? {tenant.slug}: {exc}"
                    )
```

)

File: billing\tests\test_billing_service.py

```
from datetime import date
from decimal import Decimal
from django.test import TestCase
from billing.models import PricingPlan, Invoice
from billing.services import BillingService
from tenants.models import Tenant
from usage.models import Endpoint, UsageSummary


class BillingServiceTests(TestCase):

    def setUp(self):
        self.tenant = Tenant.objects.create(name="Acme")

        self.endpoint = Endpoint.objects.create(
            tenant=self.tenant,
            name="Weather API",
            path="/weather/",
            price_per_call=Decimal("0.05"),
        )

        PricingPlan.objects.create(
            tenant=self.tenant,
            flat_monthly_fee=Decimal("10.00"),
            discount_percent=Decimal("10.00"),
        )

        UsageSummary.objects.create(
            tenant=self.tenant,
            endpoint=self.endpoint,
            period=date(2026, 3, 1),
            total_calls=100,
            total_cost=Decimal("50.00"),
        )

    def test_generate_invoice(self):

        invoice = BillingService.generate_invoice(
            tenant=self.tenant,
            period=date(2026, 3, 1),
        )

        self.assertEqual(invoice.subtotal, Decimal("60.00"))
        self.assertEqual(invoice.discount, Decimal("6.00"))
        self.assertEqual(invoice.total, Decimal("54.00"))

        self.assertEqual(Invoice.objects.count(), 1)
```

File: billing\tests\test_budget_alert.py

```
from datetime import date
from decimal import Decimal

from django.test import TestCase

from billing.models import PricingPlan, UsageAlert
from billing.services import BillingService
from tenants.models import Tenant
from usage.models import Endpoint, UsageSummary


class BudgetAlertTests(TestCase):

    def setUp(self):
        self.tenant = Tenant.objects.create(name="Acme")
        self.endpoint = Endpoint.objects.create(
            tenant=self.tenant,
            name="Weather API",
            path="/weather/",
            price_per_call=Decimal("0.05"),
        )
        PricingPlan.objects.create(
            tenant=self.tenant,
            flat_monthly_fee=Decimal("10.00"),
            discount_percent=Decimal("10.00"),
        )
        UsageSummary.objects.create(
            tenant=self.tenant,
            endpoint=self.endpoint,
            period=date(2026, 3, 1),
            total_calls=100,
            total_cost=Decimal("50.00"),
        )

    def test_budget_alert_is_triggered(self):
        UsageAlert.objects.create(
            tenant=self.tenant,
            monthly_budget=Decimal("100.00"),
            threshold_percent=80,
        )

        invoice = BillingService.generate_invoice(
            tenant=self.tenant,
            period=date(2026, 3, 1),
        )

        with self.assertLogs("billing.services") as logs:
            BillingService.check_budget_alert(invoice=invoice)

        self.assertIn("Budget alert", logs.output[0])
```

File: billing\tests\test_management_command.py

```
from datetime import date

from django.core.management import call_command
from django.test import TestCase

from billing.models import Invoice


class BillingCommandTests(TestCase):

    def test_command_does_not_create_duplicate_invoice(self):

        call_command(
            "generate_invoices",
            period="2026-03-01",
        )

        call_command(
            "generate_invoices",
            period="2026-03-01",
        )

        self.assertEqual(
            Invoice.objects.count(),
            1,
        )
```

File: core__init__.py

File: core\apps.py

```
# core/apps.py
from django.apps import AppConfig

class CoreConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'core'
```

File: core\decorators.py

```
import logging
import random
import time
from functools import wraps
from typing import Any, Callable, TypeVar

from django.db import IntegrityError, OperationalError

logger = logging.getLogger(__name__)

RT = TypeVar("RT")


def retry_on_deadlock(
    max_retries: int = 3,
    initial_backoff: float = 0.1,
    max_jitter: float = 0.05,
) -> Callable:
    """
    Retry a database operation when a deadlock or serialization failure occurs.

    Features:
    - Exponential backoff
    - Random jitter
    - Structured logging
    - Reusable decorator
    """

    def decorator(func: Callable[..., RT]) -> Callable[..., RT]:

        @wraps(func)
        def wrapper(*args: Any, **kwargs: Any) -> RT:
            last_exception = None

            for attempt in range(max_retries + 1):

                try:
                    return func(*args, **kwargs)

                except (OperationalError, IntegrityError) as exc:
                    last_exception = exc

                    message = str(exc).lower()

                    is_retryable = any(
                        error in message
                        for error in (
                            "deadlock",
                            "could not serialize",
                            "serialization failure",
                            "lock timeout",
                        )
                    )

                    if not is_retryable:
                        raise

                    if attempt >= max_retries:
                        logger.exception(
                            "Database operation failed after %s retries.",
                            max_retries,
                        )
                        raise

                    backoff = initial_backoff * (2 ** attempt)
```

```
                jitter = random.uniform(0, max_jitter)

                sleep_time = backoff + jitter

                logger.warning(
                    (
                        "Retrying database operation "
                        "(attempt %s/%s) "
                        "after %.3f seconds."
                    ),
                    attempt + 1,
                    max_retries,
                    sleep_time,
                )

                time.sleep(sleep_time)

        raise last_exception

    return wrapper

return decorator
```

File: core\exceptions.py

```
import logging

from django.core.exceptions import ValidationError as DjangoValidationError
from django.db import IntegrityError

from rest_framework import status
from rest_framework.exceptions import (
    AuthenticationFailed,
    NotAuthenticated,
    PermissionDenied,
    ValidationError,
)
from rest_framework.response import Response
from rest_framework.views import exception_handler

logger = logging.getLogger(__name__)


def custom_exception_handler(exc, context):
    """
    Global exception handler for the API.
    """

    response = exception_handler(exc, context)

    if response is not None:
        return response

    if isinstance(exc, IntegrityError):
        logger.exception("Database integrity error.")

        return Response(
            {
                "detail": (
                    "The requested operation violates a database constraint."
                )
            },
            status=status.HTTP_409_CONFLICT,
        )

    if isinstance(exc, (ValidationError, DjangoValidationError)):
        logger.warning("Validation error: %s", exc)

        return Response(
            {
                "detail": "Validation failed.",
                "errors": getattr(exc, "detail", str(exc)),
            },
            status=status.HTTP_400_BAD_REQUEST,
        )

    if isinstance(exc, AuthenticationFailed):
        logger.warning("Authentication failed.")

        return Response(
            {
                "detail": "Authentication failed."
            },
            status=status.HTTP_401_UNAUTHORIZED,
        )

    if isinstance(exc, NotAuthenticated):
        logger.warning("Authentication credentials were not provided.")

        return Response(
```

```
        {
            "detail": "Authentication credentials were not provided."
        },
        status=status.HTTP_401_UNAUTHORIZED,
    )

if isinstance(exc, PermissionDenied):
    logger.warning("Permission denied.")

    return Response(
        {
            "detail": "You do not have permission to perform this action."
        },
        status=status.HTTP_403_FORBIDDEN,
    )

logger.exception(
    "Unhandled exception.",
    exc_info=exc,
)

return Response(
    {
        "detail": "An unexpected internal server error occurred."
    },
    status=status.HTTP_500_INTERNAL_SERVER_ERROR,
)
```

File: core\local.py

```
import threading

_thread_locals = threading.local()

def get_current_request_id():
    return getattr(_thread_locals, 'request_id', None)

def set_current_request_id(request_id):
    _thread_locals.request_id = request_id
```

File: core\middleware.py

```
# core/middleware.py
import uuid
import logging
import logging
from .local import get_current_request_id

logger = logging.getLogger(__name__)

# core/middleware.py (updated)
import uuid
from .local import set_current_request_id

class RequestIDMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        request_id = request.headers.get('X-Request-ID', str(uuid.uuid4()))
        request.request_id = request_id
        set_current_request_id(request_id)
        response = self.get_response(request)
        response['X-Request-ID'] = request_id
        return response



class RequestIDFilter(logging.Filter):
    def filter(self, record):
        record.request_id = get_current_request_id()
        return True
```

File: core\utils.py

```
import logging
from contextlib import contextmanager
from typing import Generator

from django.db import connection

logger = logging.getLogger(__name__)


@contextmanager
def managed_cursor() -> Generator:
    """Context manager that provides a database cursor and ensures it is closed."""
    logger.debug("Opening a new database cursor.")
    cursor = connection.cursor()
    try:
        yield cursor
    finally:
        logger.debug("Closing the database cursor.")
        cursor.close()
```

File: core\views.py

```
import logging
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status

logger = logging.getLogger(__name__)


class HealthCheckView(APIView):
    permission_classes = []          # allow anyone
    authentication_classes = []

    def get(self, request, *args, **kwargs):
        logger.debug("Health check called")
        return Response({"status": "ok"}, status=status.HTTP_200_OK)
```

File: meterstack2__init__.py

File: meterstack2\asgi.py

```
"""
ASGI config for meterstack2 project.

It exposes the ASGI callable as a module-level variable named ``application``.

For more information on this file, see
https://docs.djangoproject.com/en/6.0/howto/deployment/asgi/
"""

import os

from django.core.asgi import get_asgi_application

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'meterstack2.settings')

application = get_asgi_application()
```

File: meterstack2\settings.py

```
"""
Django settings for meterstack2 project.
"""

from pathlib import Path
import os

# ------------------------------------------------------------------------
# Paths
# ------------------------------------------------------------------------

BASE_DIR = Path(__file__).resolve().parent.parent

# ------------------------------------------------------------------------
# Security
# ------------------------------------------------------------------------

# *** CRITICAL: no fallback ? missing env var will crash startup ***
SECRET_KEY = os.environ.get("DJANGO_SECRET_KEY", "fallback-dev-secret-key")

# *** DEBUG off by default for production safety ***
DEBUG = os.getenv("DJANGO_DEBUG", "False") == "True"

# *** Read allowed hosts from environment, default to localhost ***

ALLOWED_HOSTS = os.environ.get('DJANGO_ALLOWED_HOSTS', '127.0.0.1,localhost').split(',')

# ------------------------------------------------------------------------
# Applications
# ------------------------------------------------------------------------

INSTALLED_APPS = [
    # Django
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",

    # Third-party
    "rest_framework",
    "rest_framework_simplejwt",
    "django_filters",
    "drf_spectacular",

    # Local apps
    "tenants",
    "users",
    "usage",
    "billing",
    "core",
]

# ------------------------------------------------------------------------
# Middleware
# ------------------------------------------------------------------------

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
```

```
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
    'core.middleware.RequestIDMiddleware',
]

ROOT_URLCONF = "meterstack2.urls"

# ------------------------------------------------------------------------------
# Templates
# ------------------------------------------------------------------------------

TEMPLATES = [
    {
        "BACKEND": "django.template.backends.django.DjangoTemplates",
        "DIRS": [],
        "APP_DIRS": True,
        "OPTIONS": {
            "context_processors": [
                "django.template.context_processors.request",
                "django.contrib.auth.context_processors.auth",
                "django.contrib.messages.context_processors.messages",
            ],
        },
    },
]

WSGI_APPLICATION = "meterstack2.wsgi.application"

# ------------------------------------------------------------------------------
# Database
# ------------------------------------------------------------------------------

pg_db = os.environ.get('POSTGRES_DB')
if pg_db:
    DATABASES = {
        'default': {
            'ENGINE': 'django.db.backends.postgresql',
            'NAME': pg_db,
            'USER': os.environ.get('POSTGRES_USER', 'meterstack_user'),
            'PASSWORD': os.environ.get('POSTGRES_PASSWORD', 'change_me'),
            'HOST': os.environ.get('POSTGRES_HOST', 'db'),
            'PORT': os.environ.get('POSTGRES_PORT', '5432'),
        }
    }
else:
    DATABASES = {
        'default': {
            'ENGINE': 'django.db.backends.sqlite3',
            'NAME': BASE_DIR / 'db.sqlite3',
        }
    }

# ------------------------------------------------------------------------------
# Authentication
# ------------------------------------------------------------------------------

AUTH_USER_MODEL = "users.User"

AUTH_PASSWORD_VALIDATORS = [
    {
        "NAME": "django.contrib.auth.password_validation.UserAttributeSimilarityValidator",
    },
    {
        "NAME": "django.contrib.auth.password_validation.MinimumLengthValidator",
    },
    {
        "NAME": "django.contrib.auth.password_validation.CommonPasswordValidator",
    },
```

```
    {
        "NAME": "django.contrib.auth.password_validation.NumericPasswordValidator",
    },
]

# ------------------------------------------------------------------------------
# Internationalization
# ------------------------------------------------------------------------------

LANGUAGE_CODE = "en-us"
TIME_ZONE = "UTC"
USE_I18N = True
USE_TZ = True

# ------------------------------------------------------------------------------
# Static files
# ------------------------------------------------------------------------------

STATIC_URL = "static/"
STATIC_ROOT = BASE_DIR / "staticfiles"

# ------------------------------------------------------------------------------
# Django REST Framework
# ------------------------------------------------------------------------------

REST_FRAMEWORK = {
    "DEFAULT_AUTHENTICATION_CLASSES": (
        "rest_framework_simplejwt.authentication.JWTAuthentication",
    ),
    "DEFAULT_FILTER_BACKENDS": (
        "django_filters.rest_framework.DjangoFilterBackend",
        "rest_framework.filters.SearchFilter",
    ),
    "DEFAULT_PAGINATION_CLASS": "rest_framework.pagination.CursorPagination",
    "PAGE_SIZE": 50,
    "EXCEPTION_HANDLER": "core.exceptions.custom_exception_handler",
    "DEFAULT_SCHEMA_CLASS": "drf_spectacular.openapi.AutoSchema",

    # *** NEW: rate limiting to prevent abuse ***
    "DEFAULT_THROTTLE_CLASSES": [
        "rest_framework.throttling.AnonRateThrottle",
        "rest_framework.throttling.UserRateThrottle",
    ],
    "DEFAULT_THROTTLE_RATES": {
        "anon": "100/hour",
        "user": "1000/hour",
    },
}

# ------------------------------------------------------------------------------
# Cache
# ------------------------------------------------------------------------------

CACHES = {
    "default": {
        "BACKEND": "django.core.cache.backends.locmem.LocMemCache",
        "LOCATION": "meterstack-dashboard-cache",
        "TIMEOUT": 300,
    }
}

# ------------------------------------------------------------------------------
# Logging
# ------------------------------------------------------------------------------

LOGGING = {
    'version': 1,
```

```
    'disable_existing_loggers': False,
    'formatters': {
        'json': {
            '()': 'pythonjsonlogger.jsonlogger.JsonFormatter',
            'fmt': '%(asctime)s %(levelname)s %(name)s %(message)s',
        },
    },
    'filters': {
        'request_id': {
            '()': 'core.middleware.RequestIDFilter',   # we'll create this filter next
        },
    },
    'handlers': {
        'console': {
            'class': 'logging.StreamHandler',
            'formatter': 'json',
            'filters': ['request_id'],
        },
    },
    'root': {
        'handlers': ['console'],
        'level': 'INFO',
    },
}

# ------------------------------------------------------------------------------
# OpenAPI / Swagger
# ------------------------------------------------------------------------------

SPECTACULAR_SETTINGS = {
    "TITLE": "MeterStack API",
    "DESCRIPTION": "Multi-tenant API usage billing system",
    "VERSION": "1.0.0",
}

# ------------------------------------------------------------------------------
# Default primary key
# ------------------------------------------------------------------------------

DEFAULT_AUTO_FIELD = "django.db.models.BigAutoField"
```

File: meterstack2\urls.py

```
"""
URL configuration for meterstack2 project.
"""
from django.contrib import admin
from django.urls import path, include
from core.views import HealthCheckView

urlpatterns = [
    path('admin/', admin.site.urls),

    # JWT token endpoints
    path('api/v1/', include('users.urls')),

    # Tenant registration
    path('api/v1/', include('tenants.urls')),

    # Usage endpoints & dashboards
    path('api/v1/', include('usage.urls')),

    # Billing invoices
    path('api/v1/', include('billing.urls')),

    path('api/health/', HealthCheckView.as_view(), name='health-check'),
]
```

File: meterstack2\wsgi.py

```
"""
WSGI config for meterstack2 project.

It exposes the WSGI callable as a module-level variable named ``application``.

For more information on this file, see
https://docs.djangoproject.com/en/6.0/howto/deployment/wsgi/
"""

import os

from django.core.wsgi import get_wsgi_application

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'meterstack2.settings')

application = get_wsgi_application()
```

File: tenants__init__.py

File: tenants\admin.py

```
from django.contrib import admin
from django.contrib import admin
from .models import Tenant

@admin.register(Tenant)
class TenantAdmin(admin.ModelAdmin):
    list_display = ('name', 'slug', 'is_active', 'created_at')
    search_fields = ('name', 'slug')
    list_filter = ('is_active',)

# Register your models here.
```

File: tenants\apps.py

```
from django.apps import AppConfig


class TenantsConfig(AppConfig):
    name = 'tenants'
```

File: tenants\models.py

```
import random
import secrets
from django . db import models
from django.db import models

class Tenant(models.Model):
    name=models.CharField(max_length=300)
    slug=models.SlugField(max_length=100,db_index=True,unique=True)
    api_key=models.CharField(max_length=64,unique=True,db_index=True,editable=False)
    created_at=models.DateTimeField(auto_now_add=True)
    is_active=models.BooleanField(default=True)

    def save(self,*args,**kwargs):
        if not self.api_key:
            self.api_key=secrets.token_urlsafe(48)
        super().save(*args, **kwargs)


    def __str__(self):
        return self.name

# Create your models here.
```

File: tenants\serializers.py

```
from rest_framework import serializers
from tenants.models import Tenant
from tenants.services import create_tenant_with_admin


class TenantRegistrationSerializer(serializers.ModelSerializer):
    admin_username = serializers.CharField(write_only=True)
    admin_email = serializers.EmailField(write_only=True)
    admin_password = serializers.CharField(write_only=True, min_length=8)

    class Meta:
        model = Tenant
        fields = ['name', 'slug', 'admin_username', 'admin_email', 'admin_password']
        extra_kwargs = {'slug': {'required': False}}

    def create(self, validated_data: dict) -> Tenant:
        tenant, admin = create_tenant_with_admin(
            tenant_name=validated_data['name'],
            tenant_slug=validated_data.get('slug'),
            admin_username=validated_data['admin_username'],
            admin_email=validated_data['admin_email'],
            admin_password=validated_data['admin_password'],
        )
        return tenant
```

File: tenants\services.py

```
from django.utils.text import slugify
from tenants.models import Tenant
from users.models import User


def create_tenant_with_admin(
    *,
    tenant_name: str,
    tenant_slug: str | None = None,
    admin_username: str,
    admin_email: str,
    admin_password: str,
) -> tuple[Tenant, User]:
    """Create a new Tenant and its admin User in one atomic operation."""
    if not tenant_slug:
        tenant_slug = slugify(tenant_name)

    tenant = Tenant.objects.create(name=tenant_name, slug=tenant_slug)
    admin = User.objects.create_user(
        username=admin_username,
        email=admin_email,
        password=admin_password,
        tenant=tenant,
        role=User.Role.ADMIN,
    )
    return tenant, admin
```

File: tenants\urls.py

```
from django.urls import path
from .views import TenantRegistrationView

urlpatterns = [
    path('tenants/register/', TenantRegistrationView.as_view(), name='tenant-register'),
]
```

File: tenants\views.py

```
import logging
from rest_framework import generics, status
from rest_framework.response import Response

from tenants.models import Tenant
from tenants.serializers import TenantRegistrationSerializer

logger = logging.getLogger(__name__)


class TenantRegistrationView(generics.CreateAPIView):
    queryset = Tenant.objects.all()
    serializer_class = TenantRegistrationSerializer

    def create(self, request, *args, **kwargs) -> Response:
        try:
            serializer = self.get_serializer(data=request.data)
            serializer.is_valid(raise_exception=True)
            tenant = serializer.save()

            logger.info("Tenant registered successfully: slug=%s", tenant.slug)

            headers = self.get_success_headers(serializer.data)

            return Response(
                {
                    "message": "Tenant and admin user created successfully.",
                    "tenant": {
                        "name": tenant.name,
                        "slug": tenant.slug,
                    },
                },
                status=status.HTTP_201_CREATED,
                headers=headers,
            )
        except Exception as e:
            logger.warning("Tenant registration failed: %s", str(e))
            raise
```

File: tenants\tests__init__.py

File: tenants\tests\test_registration.py

```
from django.test import TestCase
from rest_framework import status
from rest_framework.test import APIClient

from tenants.models import Tenant
from users.models import User


class TenantRegistrationTests(TestCase):

    def setUp(self):
        self.client = APIClient()
        self.url = "/api/v1/tenants/register/"

    def test_register_tenant_creates_tenant_and_admin(self):
        data = {
            "name": "New Corp",
            "slug": "new-corp",
            "admin_username": "admin",
            "admin_email": "admin@newcorp.com",
            "admin_password": "securepass123",
        }
        response = self.client.post(self.url, data, format="json")

        self.assertEqual(response.status_code, status.HTTP_201_CREATED)
        self.assertIn("tenant", response.data)
        self.assertEqual(response.data["tenant"]["slug"], "new-corp")

        self.assertTrue(Tenant.objects.filter(slug="new-corp").exists())
        admin = User.objects.get(email="admin@newcorp.com")
        self.assertEqual(admin.role, "admin")
        self.assertEqual(admin.tenant.slug, "new-corp")
```

File: usage__init__.py

File: usage\admin.py

```
from django.contrib import admin
from django.contrib import admin
from .models import Endpoint, UsageRecord, UsageSummary

@admin.register(Endpoint)
class EndpointAdmin(admin.ModelAdmin):
    list_display = ('name', 'path', 'tenant', 'is_active', 'price_per_call')
    list_filter = ('tenant', 'is_active')
    search_fields = ('name', 'path')

@admin.register(UsageRecord)
class UsageRecordAdmin(admin.ModelAdmin):
    list_display = ('tenant', 'user', 'endpoint', 'timestamp', 'calls', 'cost')
    list_filter = ('tenant', 'endpoint', 'timestamp')
    search_fields = ('user__email',)
    date_hierarchy = 'timestamp'
    raw_id_fields = ('user',)   # for large datasets

@admin.register(UsageSummary)
class UsageSummaryAdmin(admin.ModelAdmin):
    list_display = ('tenant', 'endpoint', 'period', 'total_calls', 'total_cost')
    list_filter = ('tenant', 'period')
    search_fields = ('endpoint__name',)

# Register your models here.
```

File: usage\apps.py

```
from django.apps import AppConfig


class UsageConfig(AppConfig):
    name = 'usage'
```

File: usage\filters.py

```
import django_filters

from .models import UsageRecord


class UsageRecordFilter(django_filters.FilterSet):
    user = django_filters.NumberFilter(field_name="user__id")
    endpoint = django_filters.NumberFilter(field_name="endpoint__id")

    start_date = django_filters.DateTimeFilter(
        field_name="timestamp",
        lookup_expr="gte",
    )

    end_date = django_filters.DateTimeFilter(
        field_name="timestamp",
        lookup_expr="lte",
    )

    class Meta:
        model = UsageRecord
        fields = [
            "user",
            "endpoint",
            "start_date",
            "end_date",
        ]
```

File: usage\managers.py

```
from __future__ import annotations
from datetime import date, timedelta
from decimal import Decimal
from typing import Any
from django.db import models
from django.db.models import Sum
from django.db.models.functions import TruncDate
from tenants.models import Tenant


class UsageRecordManager(models.Manager):
    """
    Query helpers for raw UsageRecord data.
    """

    def daily_trends(
        self,
        *,
        tenant: Tenant,
        days: int = 30,
    ) -> list[dict[str, Any]]:
        """
        Return daily usage statistics for the last N days.
        """

        start_date = date.today() - timedelta(days=days)

        return list(
            self.filter(
                tenant=tenant,
                timestamp__date__gte=start_date,
            )
            .annotate(
                day=TruncDate("timestamp"),
            )
            .values(
                "day",
            )
            .annotate(
                total_calls=Sum("calls"),
                total_cost=Sum("cost"),
            )
            .order_by("day")
        )

    def current_month_usage(
        self,
        *,
        tenant: Tenant,
    ) -> dict[str, Any]:
        """
        Return current month's total usage directly
        from UsageRecord.
        """

        current_period = date.today().replace(day=1)

        result = self.filter(
            tenant=tenant,
            timestamp__date__gte=current_period,
        ).aggregate(
            total_calls=Sum("calls"),
            total_cost=Sum("cost"),
            total_bytes=Sum("data_bytes"),
        )
```

```
        return {
            "total_calls": result["total_calls"] or 0,
            "total_cost": result["total_cost"] or Decimal("0"),
            "total_bytes": result["total_bytes"] or 0,
        }


class UsageSummaryManager(models.Manager):
    """
    Query helpers for aggregated monthly usage.
    """

    def top_endpoints(
        self,
        *,
        tenant: Tenant,
        limit: int = 5,
    ) -> list[dict[str, Any]]:
        """
        Return the most used endpoints.
        """

        return list(
            self.filter(
                tenant=tenant,
            )
            .values(
                "endpoint__name",
            )
            .annotate(
                total_calls=Sum("total_calls"),
                total_cost=Sum("total_cost"),
            )
            .order_by("-total_calls")[:limit]
        )

    def compare_periods(
        self,
        *,
        tenant: Tenant,
        current_period: date,
        previous_period: date,
    ) -> dict[str, Any]:
        """
        Compare two billing periods.
        """

        current = (
            self.filter(
                tenant=tenant,
                period=current_period,
            )
            .aggregate(
                total_calls=Sum("total_calls"),
                total_cost=Sum("total_cost"),
                total_bytes=Sum("total_bytes"),
            )
        )

        previous = (
            self.filter(
                tenant=tenant,
                period=previous_period,
            )
            .aggregate(
                total_calls=Sum("total_calls"),
```

```
        total_cost=Sum("total_cost"),
        total_bytes=Sum("total_bytes"),
    )
)

return {
    "current": {
        "total_calls": current["total_calls"] or 0,
        "total_cost": current["total_cost"] or Decimal("0"),
        "total_bytes": current["total_bytes"] or 0,
    },
    "previous": {
        "total_calls": previous["total_calls"] or 0,
        "total_cost": previous["total_cost"] or Decimal("0"),
        "total_bytes": previous["total_bytes"] or 0,
    },
}
```

File: usage\models.py

```
from django.conf import settings
from django.db import models
from tenants.models import Tenant
from .managers import (
    UsageRecordManager,
    UsageSummaryManager,
)
from decimal import Decimal
from django.core.validators import MinValueValidator

class Endpoint(models.Model):
    tenant = models.ForeignKey(
        Tenant,
        on_delete=models.CASCADE,
        related_name="endpoints",
        db_index=True,
    )

    name = models.CharField(max_length=200)

    path = models.CharField(max_length=500)

    price_per_call = models.DecimalField(
        max_digits=14,
        decimal_places=6,
        default=0,
        validators=[MinValueValidator(Decimal("0.00"))],
    )

    price_per_kb = models.DecimalField(
        max_digits=14,
        decimal_places=6,
        default=0,
        validators=[MinValueValidator(Decimal("0.00"))],
    )

    is_active = models.BooleanField(default=True)

    created_at = models.DateTimeField(
        auto_now_add=True,
    )

    class Meta:
        constraints = [
            models.UniqueConstraint(
                fields=["tenant", "path"],
                name="unique_endpoint_per_tenant",
            ),
            models.CheckConstraint(
                condition=models.Q(price_per_call__gte=0) & models.Q(price_per_kb__gte=0),
                name="endpoint_prices_non_negative",
            ),
        ]

        indexes = [
            models.Index(fields=["tenant", "is_active"]),
            models.Index(fields=["tenant", "path"]),
        ]

        ordering = ["name"]

    def __str__(self):
        return f"{self.tenant.slug} - {self.name}"
```

```
class UsageRecord(models.Model):
    tenant = models.ForeignKey(
        Tenant,
        on_delete=models.CASCADE,
        related_name="usage_records",
        db_index=True,
    )

    user = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.SET_NULL,
        null=True,
        blank=True,
        related_name="usage_records",
    )

    endpoint = models.ForeignKey(
        Endpoint,
        on_delete=models.SET_NULL,
        null=True,
        blank=True,
        related_name="usage_records",
    )

    timestamp = models.DateTimeField(
        auto_now_add=True,
        db_index=True,
    )

    calls = models.PositiveIntegerField(default=1)

    data_bytes = models.PositiveBigIntegerField(default=0)

    cost = models.DecimalField(
        max_digits=12,
        decimal_places=8,
        default=0,
    )

    # Custom manager
    objects = UsageRecordManager()

    class Meta:
        indexes = [
            models.Index(fields=["tenant", "timestamp"]),
            models.Index(fields=["tenant", "user"]),
            models.Index(fields=["tenant", "endpoint"]),
            models.Index(fields=["tenant", "timestamp", "endpoint"]),
        ]

        constraints = [
            models.CheckConstraint(
                condition=(
                    models.Q(calls__gte=0)
                    & models.Q(data_bytes__gte=0)
                    & models.Q(cost__gte=0)
                ),
                name="usage_record_positive_fields",
            ),
        ]

        ordering = ["-timestamp"]

    def __str__(self):
        return (
            f"{self.tenant.slug} | "
            f"{self.user} | "
```

```
            f"{self.endpoint} | "
            f"{self.timestamp}"
        )

class UsageSummary(models.Model):
    tenant = models.ForeignKey(
        Tenant,
        on_delete=models.CASCADE,
        related_name="usage_summaries",
    )

    endpoint = models.ForeignKey(
        Endpoint,
        on_delete=models.CASCADE,
        related_name="usage_summaries",
    )

    period = models.DateField()

    total_calls = models.PositiveIntegerField(default=0)

    total_bytes = models.PositiveBigIntegerField(default=0)

    total_cost = models.DecimalField(
        max_digits=12,
        decimal_places=8,
        default=0,
    )

    # Custom manager
    objects = UsageSummaryManager()

    class Meta:
        constraints = [
            models.UniqueConstraint(
                fields=[
                    "tenant",
                    "endpoint",
                    "period",
                ],
                name="unique_usage_summary_per_period",
            ),
            # NEW: non?negative guard
            models.CheckConstraint(
                condition=(
                    models.Q(total_calls__gte=0)
                    & models.Q(total_bytes__gte=0)
                    & models.Q(total_cost__gte=0)
                ),
                name="usage_summary_positive_fields",
            ),
        ]

        indexes = [
            models.Index(fields=["tenant", "period"]),
            models.Index(fields=["tenant", "endpoint"]),
        ]

        ordering = ["-period"]

    def __str__(self):
        return (
            f"{self.tenant.slug} - "
            f"{self.endpoint.name} - "
            f"{self.period}"
        )
```

File: usage\pagination.py

```
from rest_framework.pagination import CursorPagination


class UsageCursorPagination(CursorPagination):
    ordering = "-timestamp"
    page_size = 50
    page_size_query_param = "page_size"
    max_page_size = 200
```

File: usage\serializers.py

```
from rest_framework import serializers
from rest_framework import serializers
from .models import UsageRecord



class DataRequestSerializer(serializers.Serializer):
    endpoint_path = serializers.CharField(max_length=500)




class UsageRecordSerializer(serializers.ModelSerializer):
    user_email = serializers.EmailField(
        source="user.email",
        read_only=True,
    )

    endpoint_name = serializers.CharField(
        source="endpoint.name",
        read_only=True,
    )

    class Meta:
        model = UsageRecord
        fields = [
            "id",
            "user_email",
            "endpoint_name",
            "timestamp",
            "calls",
            "data_bytes",
            "cost",
        ]


from rest_framework import serializers


class TopEndpointSerializer(serializers.Serializer):
    endpoint__name = serializers.CharField()

    total_calls = serializers.IntegerField()

    total_cost = serializers.DecimalField(
        max_digits=12,
        decimal_places=8,
    )


class PeriodSummarySerializer(serializers.Serializer):
    total_calls = serializers.IntegerField()

    total_cost = serializers.DecimalField(
        max_digits=12,
        decimal_places=8,
    )

    total_bytes = serializers.IntegerField()


class PeriodComparisonSerializer(serializers.Serializer):
    current = PeriodSummarySerializer()

    previous = PeriodSummarySerializer()
```

```
class DashboardSummarySerializer(serializers.Serializer):
    top_endpoints = TopEndpointSerializer(
        many=True,
    )

    comparison = PeriodComparisonSerializer()


class DailyTrendSerializer(serializers.Serializer):
    day = serializers.DateField()

    total_calls = serializers.IntegerField()

    total_cost = serializers.DecimalField(
        max_digits=12,
        decimal_places=8,
    )


class DashboardTrendSerializer(serializers.Serializer):
    trends = DailyTrendSerializer(
        many=True,
    )
```

File: usage\urls.py

```
from django.urls import path

from .views import (
    DataAPIView,
    UsageListAPIView,
    DashboardSummaryAPIView,
    DashboardTrendsAPIView,
)

urlpatterns = [
    path(
        "data/",
        DataAPIView.as_view(),
        name="data-endpoint",
    ),

    path(
        "usage/",
        UsageListAPIView.as_view(),
        name="usage-list",
    ),

    path(
        "dashboard/summary/",
        DashboardSummaryAPIView.as_view(),
        name="dashboard-summary",
    ),

    path(
        "dashboard/trends/",
        DashboardTrendsAPIView.as_view(),
        name="dashboard-trends",
    ),
]
```

File: usage\views.py

```
import logging
from datetime import date

from dateutil.relativedelta import relativedelta
from django_filters.rest_framework import DjangoFilterBackend
from drf_spectacular.utils import (
    OpenApiExample,
    OpenApiResponse,
    extend_schema,
)
from rest_framework import status
from rest_framework.filters import SearchFilter
from rest_framework.generics import ListAPIView
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response
from rest_framework.views import APIView

from users.permissions import IsTenantAdmin
from .filters import UsageRecordFilter
from .pagination import UsageCursorPagination
from .serializers import (
    DataRequestSerializer,
    UsageRecordSerializer,
    DashboardSummarySerializer,
    DashboardTrendSerializer,
)
from .services.usage import UsageService          # adjust if your package layout differs
from .services.reporting import ReportingService

logger = logging.getLogger(__name__)


class DataAPIView(APIView):
    """
    Public endpoint that tenants call to receive data.
    Every successful request is automatically metered.
    """
    permission_classes = [IsAuthenticated]

    @extend_schema(
        summary="Retrieve data from an API endpoint",
        description=(
            "Returns sample data and automatically records "
            "API usage for billing."
        ),
        request=DataRequestSerializer,
        responses={
            200: OpenApiResponse(description="Data returned successfully"),
            400: OpenApiResponse(description="Validation error"),
            404: OpenApiResponse(description="Endpoint not found"),
        },
        examples=[
            OpenApiExample(
                "Example Request",
                value={"endpoint_path": "/weather/"},
                request_only=True,
            )
        ],
        tags=["Usage"],
    )
    def post(self, request):
        serializer = DataRequestSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)

        endpoint = UsageService.get_active_endpoint(
```

```
            tenant=request.user.tenant,
            endpoint_path=serializer.validated_data["endpoint_path"],
        )

        payload = {
            "temperature": 22,
            "humidity": 65,
            "status": "Sunny",
        }

        # Business rule moved to service layer
        data_bytes = UsageService.get_data_size(payload)

        UsageService.record_usage(
            tenant=request.user.tenant,
            user=request.user,
            endpoint=endpoint,
            calls=1,
            data_bytes=data_bytes,
        )

        # Dashboard data changed ? clear relevant caches
        ReportingService.invalidate_dashboard_cache(tenant=request.user.tenant)

        logger.info(
            "Data served | tenant=%s endpoint=%s user=%s",
            request.user.tenant.slug,
            endpoint.path,
            request.user.email,
        )

        return Response(
            {
                "endpoint": endpoint.name,
                "data": payload,
            },
            status=status.HTTP_200_OK,
        )


@extend_schema(
    summary="Usage dashboard",
    description="List usage records for the current tenant.",
    tags=["Usage"],
)
class UsageListAPIView(ListAPIView):
    serializer_class = UsageRecordSerializer
    permission_classes = [IsAuthenticated, IsTenantAdmin]
    pagination_class = UsageCursorPagination
    filter_backends = [DjangoFilterBackend, SearchFilter]
    filterset_class = UsageRecordFilter
    search_fields = ["user__email"]
    ordering = ["-timestamp"]

    def get_queryset(self):
        return UsageService.get_filtered_usage(tenant=self.request.user.tenant)


@extend_schema(
    summary="Dashboard summary",
    description="Return dashboard summary for the current tenant.",
    tags=["Dashboard"],
)
class DashboardSummaryAPIView(APIView):
    permission_classes = [IsAuthenticated, IsTenantAdmin]

    def get(self, request):
```

```
        tenant = request.user.tenant
        current_period = date.today().replace(day=1)
        previous_period = current_period - relativedelta(months=1)

        data = {
            "top_endpoints": ReportingService.get_top_endpoints(tenant=tenant),
            "comparison": ReportingService.compare_periods(
                tenant=tenant,
                current_period=current_period,
                previous_period=previous_period,
            ),
        }
        serializer = DashboardSummarySerializer(data)
        return Response(serializer.data)


@extend_schema(
    summary="Dashboard trends",
    description="Return daily usage trends.",
    tags=["Dashboard"],
)
class DashboardTrendsAPIView(APIView):
    permission_classes = [IsAuthenticated, IsTenantAdmin]

    def get(self, request):
        tenant = request.user.tenant
        data = {
            "trends": ReportingService.get_daily_usage_trends(tenant=tenant)
        }
        serializer = DashboardTrendSerializer(data)
        return Response(serializer.data)
```

File: usage\services__init__.py

File: usage\services\reporting.py

```
from __future__ import annotations
import logging
import time
from datetime import date
from typing import Any, Callable
from django.core.cache import cache
from tenants.models import Tenant
from usage.models import UsageRecord, UsageSummary
from datetime import date
from dateutil.relativedelta import relativedelta

logger = logging.getLogger(__name__)


class ReportingService:
    """
    Provides reporting queries used by dashboard APIs.
    """

    SLOW_QUERY_THRESHOLD = 0.5

    CACHE_TIMEOUT = 300
    CACHE_PREFIX = "dashboard"

    @classmethod
    def _log_query_time(
        cls,
        query_name: str,
        started_at: float,
    ) -> None:
        """
        Log slow queries.
        """

        elapsed = time.perf_counter() - started_at

        if elapsed >= cls.SLOW_QUERY_THRESHOLD:
            logger.warning(
                "%s took %.3f seconds",
                query_name,
                elapsed,
            )

    @classmethod
    def _get_or_set_cache(
        cls,
        *,
        key: str,
        callback: Callable[[], Any],
    ) -> Any:
        """
        Cache-Aside helper.
        """

        cached_data = cache.get(key)

        if cached_data is not None:
            logger.info(
                "Cache HIT | %s",
                key,
            )
            return cached_data

        logger.info(
            "Cache MISS | %s",
```

```
        key,
    )

    result = callback()

    cache.set(
        key,
        result,
        timeout=cls.CACHE_TIMEOUT,
    )

    return result

@classmethod
def invalidate_dashboard_cache(
    cls,
    tenant: Tenant,
) -> None:
    """
    Remove all cached dashboard data for a tenant.

    Called whenever usage data changes so the next dashboard
    request is rebuilt from the database.
    """

    current_period = date.today().replace(day=1)
    previous_period = current_period - relativedelta(months=1)

    keys = [
        (
            f"{cls.CACHE_PREFIX}:"
            f"top_endpoints:"
            f"{tenant.slug}:5"
        ),
        (
            f"{cls.CACHE_PREFIX}:"
            f"daily_trends:"
            f"{tenant.slug}:30"
        ),
        (
            f"{cls.CACHE_PREFIX}:"
            f"compare_periods:"
            f"{tenant.slug}:"
            f"{current_period}:"
            f"{previous_period}"
        ),
    ]

    cache.delete_many(keys)

    logger.info(
        "Dashboard cache invalidated | tenant=%s",
        tenant.slug,
    )

@classmethod
def get_top_endpoints(
    cls,
    tenant: Tenant,
    limit: int = 5,
) -> list[dict[str, Any]]:
    """
    Return the most used endpoints.
    """

    started = time.perf_counter()
```

```
        cache_key = (
            f"{cls.CACHE_PREFIX}:"
            f"top_endpoints:"
            f"{tenant.slug}:"
            f"{limit}"
        )

        def query():
            return UsageSummary.objects.top_endpoints(
                tenant=tenant,
                limit=limit,
            )

        result = cls._get_or_set_cache(
            key=cache_key,
            callback=query,
        )

        cls._log_query_time(
            "get_top_endpoints",
            started,
        )

        return result

    @classmethod
    def get_daily_usage_trends(
        cls,
        tenant: Tenant,
        days: int = 30,
    ) -> list[dict[str, Any]]:
        """
        Return daily usage trends.
        """

        started = time.perf_counter()

        cache_key = (
            f"{cls.CACHE_PREFIX}:"
            f"daily_trends:"
            f"{tenant.slug}:"
            f"{days}"
        )

        def query():
            return UsageRecord.objects.daily_trends(
                tenant=tenant,
                days=days,
            )

        result = cls._get_or_set_cache(
            key=cache_key,
            callback=query,
        )

        cls._log_query_time(
            "get_daily_usage_trends",
            started,
        )

        return result

    @classmethod
    def compare_periods(
        cls,
        tenant: Tenant,
        current_period: date,
```

```
    previous_period: date,
) -> dict[str, Any]:
    """
    Compare two reporting periods.
    """

    started = time.perf_counter()

    cache_key = (
        f"{cls.CACHE_PREFIX}:"
        f"compare_periods:"
        f"{tenant.slug}:"
        f"{current_period}:"
        f"{previous_period}"
    )

    def query():
        return UsageSummary.objects.compare_periods(
            tenant=tenant,
            current_period=current_period,
            previous_period=previous_period,
        )

    result = cls._get_or_set_cache(
        key=cache_key,
        callback=query,
    )

    cls._log_query_time(
        "compare_periods",
        started,
    )

    return result
```

File: usage\services\usage.py

```
import logging
from datetime import date
from decimal import Decimal

from django.db import IntegrityError, transaction
from django.db.models import F, QuerySet
from django.shortcuts import get_object_or_404

from core.decorators import retry_on_deadlock
from tenants.models import Tenant
from usage.models import Endpoint, UsageRecord, UsageSummary

logger = logging.getLogger(__name__)


class UsageService:
    @staticmethod
    def calculate_cost(
        *,
        endpoint: Endpoint,
        calls: int,
        data_bytes: int,
    ) -> Decimal:
        """Calculate the cost of a single usage event."""
        data_kb = Decimal(data_bytes) / Decimal(1024)
        return (
            endpoint.price_per_call * calls
            + endpoint.price_per_kb * data_kb
        )

    @staticmethod
    def get_current_period():
        """Return the first day of the current month (e.g. 2026-07-01)."""
        today = date.today()
        return today.replace(day=1)

    @staticmethod
    def get_data_size(payload: dict) -> int:
        """
        Return the byte size of a payload dictionary as it would be
        transmitted over the wire.
        """
        return len(str(payload).encode("utf-8"))

    @staticmethod
    @retry_on_deadlock()
    @transaction.atomic
    def update_usage_summary(
        *,
        tenant: Tenant,
        endpoint: Endpoint,
        calls: int,
        data_bytes: int,
        cost: Decimal,
    ) -> UsageSummary:
        """
        Thread?safe update of the monthly usage summary.
        Uses an atomic update-first approach to avoid race conditions.
        """
        period = UsageService.get_current_period()

        # Try to update an existing row
        updated = (
            UsageSummary.objects
            .select_for_update()
```

```
        .filter(
            tenant=tenant,
            endpoint=endpoint,
            period=period,
        )
        .update(
            total_calls=F("total_calls") + calls,
            total_bytes=F("total_bytes") + data_bytes,
            total_cost=F("total_cost") + cost,
        )
    )

    if updated == 0:
        # Row didn't exist ? create it.
        summary, created = UsageSummary.objects.get_or_create(
            tenant=tenant,
            endpoint=endpoint,
            period=period,
            defaults={
                "total_calls": calls,
                "total_bytes": data_bytes,
                "total_cost": cost,
            },
        )

        if not created:
            # Another thread created it while we were trying;
            # update the row we just fetched with a lock
            summary = (
                UsageSummary.objects
                .select_for_update()
                .get(pk=summary.pk)
            )
            summary.total_calls = F("total_calls") + calls
            summary.total_bytes = F("total_bytes") + data_bytes
            summary.total_cost = F("total_cost") + cost
            summary.save(update_fields=["total_calls", "total_bytes", "total_cost"])

    # Always return the final row
    summary = UsageSummary.objects.get(
        tenant=tenant,
        endpoint=endpoint,
        period=period,
    )
    return summary

@staticmethod
def create_usage_record(
    *,
    tenant: Tenant,
    user,
    endpoint: Endpoint,
    calls: int,
    data_bytes: int,
    cost: Decimal,
) -> UsageRecord:
    """Store one immutable usage record."""
    return UsageRecord.objects.create(
        tenant=tenant,
        user=user,
        endpoint=endpoint,
        calls=calls,
        data_bytes=data_bytes,
        cost=cost,
    )

@staticmethod
```

```
def record_usage(
    *,
    tenant: Tenant,
    user,
    endpoint: Endpoint,
    calls: int = 1,
    data_bytes: int = 0,
) -> UsageRecord:
    """
    Record one API usage event.

    Workflow:
    1. Calculate cost
    2. Update monthly summary
    3. Create immutable usage record

    (Cache invalidation is now handled by the calling view.)
    """
    cost = UsageService.calculate_cost(
        endpoint=endpoint,
        calls=calls,
        data_bytes=data_bytes,
    )

    UsageService.update_usage_summary(
        tenant=tenant,
        endpoint=endpoint,
        calls=calls,
        data_bytes=data_bytes,
        cost=cost,
    )

    usage_record = UsageService.create_usage_record(
        tenant=tenant,
        user=user,
        endpoint=endpoint,
        calls=calls,
        data_bytes=data_bytes,
        cost=cost,
    )

    logger.info(
        "Usage recorded | tenant=%s endpoint=%s user=%s calls=%s bytes=%s cost=%s",
        tenant.slug,
        endpoint.path,
        getattr(user, "email", None),
        calls,
        data_bytes,
        cost,
    )

    return usage_record

@staticmethod
def get_active_endpoint(*, tenant, endpoint_path: str) -> Endpoint:
    """Return an active endpoint for the given tenant."""
    return get_object_or_404(
        Endpoint,
        tenant=tenant,
        path=endpoint_path,
        is_active=True,
    )

@staticmethod
def get_filtered_usage(*, tenant) -> QuerySet:
    """
    Base queryset for the usage dashboard.
```

```
    Filtering, searching and pagination are handled by DRF backends.
    """
    return (
        UsageRecord.objects
        .filter(tenant=tenant)
        .select_related("user", "endpoint")
        .order_by("-timestamp")
    )
```

File: usage\tests\test_concurrency.py

```
import concurrent.futures
from decimal import Decimal

from django.test import TransactionTestCase
from django.utils import timezone

from tenants.models import Tenant
from users.models import User

from usage.models import Endpoint, UsageRecord, UsageSummary
from usage.services import UsageService


class UsageConcurrencyTests(TransactionTestCase):
    """
    Verify that concurrent requests never lose or double-count usage.
    """

    reset_sequences = True

    @classmethod
    def setUpTestData(cls):
        cls.tenant = Tenant.objects.create(
            name="Acme Inc.",
            slug="acme",
            api_key="test-api-key",
        )

        cls.user = User.objects.create_user(
            username="john",
            email="john@example.com",
            password="password123",
            tenant=cls.tenant,
            role="member",
        )

        cls.endpoint = Endpoint.objects.create(
            tenant=cls.tenant,
            name="Weather API",
            path="/weather/",
            price_per_call=Decimal("0.01000000"),
            price_per_kb=Decimal("0.00100000"),
        )

    def test_concurrent_usage_recording(self):
        """
        Simulate many users calling the API simultaneously.
        """

        total_requests = 50
        bytes_per_request = 1024

        with concurrent.futures.ThreadPoolExecutor(
            max_workers=20
        ) as executor:

            futures = [
                executor.submit(
                    UsageService.record_usage,
                    tenant=self.tenant,
                    user=self.user,
                    endpoint=self.endpoint,
                    calls=1,
                    data_bytes=bytes_per_request,
                )
```

```
            for _ in range(total_requests)
        ]

        for future in futures:
            future.result()

    self.assertEqual(
        UsageRecord.objects.count(),
        total_requests,
    )

    period = timezone.now().date().replace(day=1)

    summary = UsageSummary.objects.get(
        tenant=self.tenant,
        endpoint=self.endpoint,
        period=period,
    )

    self.assertEqual(
        summary.total_calls,
        total_requests,
    )

    self.assertEqual(
        summary.total_bytes,
        total_requests * bytes_per_request,
    )

    expected_cost_per_request = (
        self.endpoint.price_per_call
        + self.endpoint.price_per_kb
        * Decimal(bytes_per_request)
        / Decimal(1024)
    )

    expected_total_cost = (
        expected_cost_per_request * total_requests
    )

    self.assertEqual(
        summary.total_cost,
        expected_total_cost,
    )

def test_every_usage_record_is_created(self):
    """
    Every request should create exactly one UsageRecord.
    """

    request_count = 25

    with concurrent.futures.ThreadPoolExecutor(
        max_workers=10
    ) as executor:

        futures = [
            executor.submit(
                UsageService.record_usage,
                tenant=self.tenant,
                user=self.user,
                endpoint=self.endpoint,
                data_bytes=512,
            )
            for _ in range(request_count)
        ]
```

```
        for future in futures:
            future.result()

    records = UsageRecord.objects.filter(
        tenant=self.tenant,
        endpoint=self.endpoint,
    )

    self.assertEqual(records.count(), request_count)

    for record in records:
        self.assertEqual(record.calls, 1)
        self.assertEqual(record.data_bytes, 512)
        self.assertIsNotNone(record.cost)
        self.assertEqual(record.user, self.user)
        self.assertEqual(record.endpoint, self.endpoint)

def test_only_one_summary_row_is_created(self):
    """
    Concurrent requests should never create duplicate monthly summaries.
    """

    with concurrent.futures.ThreadPoolExecutor(
        max_workers=30
    ) as executor:

        futures = [
            executor.submit(
                UsageService.record_usage,
                tenant=self.tenant,
                user=self.user,
                endpoint=self.endpoint,
                data_bytes=100,
            )
            for _ in range(100)
        ]

        for future in futures:
            future.result()

    period = timezone.now().date().replace(day=1)

    self.assertEqual(
        UsageSummary.objects.filter(
            tenant=self.tenant,
            endpoint=self.endpoint,
            period=period,
        ).count(),
        1,
    )
```

File: usage\tests\test_data_api.py

```
from decimal import Decimal
from django.test import TestCase
from rest_framework import status
from rest_framework.test import APIClient
from tenants.models import Tenant
from usage.models import Endpoint, UsageRecord, UsageSummary
from users.models import User


class DataAPITests(TestCase):

    def setUp(self):
        self.client = APIClient()
        self.tenant = Tenant.objects.create(name="Acme", slug="acme")
        self.user = User.objects.create_user(
            username="testuser",
            email="test@acme.com",
            password="testpass123",
            tenant=self.tenant,
            role="member",
        )
        self.endpoint = Endpoint.objects.create(
            tenant=self.tenant,
            name="Weather",
            path="/weather/",
            price_per_call=Decimal("0.01"),
            price_per_kb=Decimal("0.001"),
        )
        # Obtain token
        token_resp = self.client.post(
            "/api/v1/token/",
            {"username": "testuser", "password": "testpass123"},
            format="json",
        )
        self.token = token_resp.data["access"]
        self.client.credentials(HTTP_AUTHORIZATION=f"Bearer {self.token}")

    def test_data_endpoint_records_usage(self):
        response = self.client.post(
            "/api/v1/data/",
            {"endpoint_path": "/weather/"},
            format="json",
        )
        self.assertEqual(response.status_code, status.HTTP_200_OK)
        self.assertIn("data", response.data)

        # Verify usage record created
        self.assertEqual(UsageRecord.objects.count(), 1)
        record = UsageRecord.objects.first()
        self.assertEqual(record.calls, 1)
        self.assertGreater(record.data_bytes, 0)
        self.assertGreater(record.cost, Decimal("0"))

        # Verify monthly summary updated
        period = record.timestamp.date().replace(day=1)
        summary = UsageSummary.objects.get(
            tenant=self.tenant,
            endpoint=self.endpoint,
            period=period,
        )
        self.assertEqual(summary.total_calls, 1)

    def test_unauthenticated_request(self):
        self.client.credentials()  # remove token
        response = self.client.post(
```

```
    "/api/v1/data/",
    {"endpoint_path": "/weather/"},
    format="json",
)
self.assertEqual(response.status_code, status.HTTP_401_UNAUTHORIZED)
```

File: usage\tests\test_reporting_service.py

```
from datetime import date, timedelta
from decimal import Decimal
from django.core.cache import cache
from django.test import TestCase
from tenants.models import Tenant
from usage.models import Endpoint, UsageRecord, UsageSummary
from usage.services.reporting import ReportingService


class ReportingServiceTests(TestCase):

    def setUp(self):

        cache.clear()

        self.tenant = Tenant.objects.create(
            name="Acme",
            slug="acme",
        )

        self.endpoint1 = Endpoint.objects.create(
            tenant=self.tenant,
            name="Weather",
            path="/weather/",
            price_per_call=Decimal("0.10"),
            price_per_kb=Decimal("0.01"),
        )

        self.endpoint2 = Endpoint.objects.create(
            tenant=self.tenant,
            name="News",
            path="/news/",
            price_per_call=Decimal("0.20"),
            price_per_kb=Decimal("0.02"),
        )

        self.current_period = date.today().replace(day=1)

        self.previous_period = (
            self.current_period - timedelta(days=30)
        ).replace(day=1)

        UsageSummary.objects.create(
            tenant=self.tenant,
            endpoint=self.endpoint1,
            period=self.current_period,
            total_calls=500,
            total_bytes=50000,
            total_cost=Decimal("55.50"),
        )

        UsageSummary.objects.create(
            tenant=self.tenant,
            endpoint=self.endpoint2,
            period=self.current_period,
            total_calls=200,
            total_bytes=20000,
            total_cost=Decimal("22.20"),
        )

        UsageSummary.objects.create(
            tenant=self.tenant,
            endpoint=self.endpoint1,
            period=self.previous_period,
            total_calls=100,
```

```
            total_bytes=10000,
            total_cost=Decimal("11.10"),
        )

        UsageRecord.objects.create(
            tenant=self.tenant,
            endpoint=self.endpoint1,
            calls=20,
            data_bytes=2000,
            cost=Decimal("2.20"),
        )

        UsageRecord.objects.create(
            tenant=self.tenant,
            endpoint=self.endpoint1,
            calls=30,
            data_bytes=3000,
            cost=Decimal("3.30"),
        )

        UsageRecord.objects.create(
            tenant=self.tenant,
            endpoint=self.endpoint2,
            calls=10,
            data_bytes=1000,
            cost=Decimal("1.10"),
        )

        self.top_endpoint_cache_key = (
            f"{ReportingService.CACHE_PREFIX}:"
            f"top_endpoints:"
            f"{self.tenant.slug}:5"
        )

    def test_get_top_endpoints(self):

        result = ReportingService.get_top_endpoints(
            tenant=self.tenant,
        )

        self.assertEqual(
            len(result),
            2,
        )

        self.assertEqual(
            result[0]["endpoint__name"],
            "Weather",
        )

        self.assertEqual(
            result[0]["total_calls"],
            600,
        )

        self.assertEqual(
            result[1]["endpoint__name"],
            "News",
        )

        self.assertEqual(
            result[1]["total_calls"],
            200,
        )

    def test_get_daily_usage_trends(self):
```

```
        result = ReportingService.get_daily_usage_trends(
            tenant=self.tenant,
            days=30,
        )

        self.assertEqual(
            len(result),
            1,
        )

        self.assertEqual(
            result[0]["total_calls"],
            60,
        )

        self.assertEqual(
            result[0]["total_cost"],
            Decimal("6.60"),
        )

    def test_compare_periods(self):

        result = ReportingService.compare_periods(
            tenant=self.tenant,
            current_period=self.current_period,
            previous_period=self.previous_period,
        )

        self.assertEqual(
            result["current"]["total_calls"],
            700,
        )

        self.assertEqual(
            result["previous"]["total_calls"],
            100,
        )

        self.assertEqual(
            result["current"]["total_cost"],
            Decimal("77.70"),
        )

        self.assertEqual(
            result["previous"]["total_cost"],
            Decimal("11.10"),
        )

    def test_dashboard_cache_created(self):

        cache.clear()

        self.assertIsNone(
            cache.get(self.top_endpoint_cache_key),
        )

        ReportingService.get_top_endpoints(
            tenant=self.tenant,
        )

        self.assertIsNotNone(
            cache.get(self.top_endpoint_cache_key),
        )

    def test_dashboard_cache_hit(self):

        cache.clear()
```

```
    first_result = ReportingService.get_top_endpoints(
        tenant=self.tenant,
    )

    second_result = ReportingService.get_top_endpoints(
        tenant=self.tenant,
    )

    self.assertEqual(
        first_result,
        second_result,
    )

    self.assertIsNotNone(
        cache.get(self.top_endpoint_cache_key),
    )


def test_dashboard_cache_invalidated(self):
        cache.clear()
        ReportingService.get_top_endpoints(
            tenant=self.tenant,
        )
        self.assertIsNotNone(
            cache.get(self.top_endpoint_cache_key),
        )

        ReportingService.invalidate_dashboard_cache(
            tenant=self.tenant,
        )
        self.assertIsNone(
            cache.get(self.top_endpoint_cache_key),
        )
```

File: users__init__.py

File: users\admin.py

```
from django.contrib import admin

# Register your models here.
```

File: users\apps.py

```
from django.apps import AppConfig


class UsersConfig(AppConfig):
    name = 'users'
```

File: users\authentication.py

```
from rest_framework_simplejwt.authentication import JWTAuthentication
from rest_framework.exceptions import AuthenticationFailed
from django.contrib.auth import get_user_model
from rest_framework_simplejwt.settings import api_settings

User = get_user_model()

class CustomJWTAuthentication(JWTAuthentication):
    def get_user(self, validated_token):
        """
        Override to always select the tenant when fetching the user.
        This eliminates an extra query later.
        """
        try:
            user_id = validated_token[api_settings.USER_ID_CLAIM]
        except KeyError:
            raise AuthenticationFailed('Token contained no recognizable user identification')

        try:
            user = User.objects.select_related('tenant').get(**{api_settings.USER_ID_FIELD: user_id})
        except User.DoesNotExist:
            raise AuthenticationFailed('User not found')

        return user

    def authenticate(self, request):
        result = super().authenticate(request)
        if result is None:
            return None

        user, validated_token = result
        tenant_slug = validated_token.get('tenant_slug')
        if not tenant_slug:
            raise AuthenticationFailed('Token does not contain a tenant slug')

        # Tenant already preloaded via select_related, so request.user.tenant is cached
        return (user, validated_token)
```

File: users\models.py

```
from django.db import models
from django.db.models import TextChoices
from tenants.models import Tenant
from django . contrib . auth . models import AbstractUser


class User(AbstractUser):
    class Role(TextChoices):
        admin='admin','admin'
        member='member','member'
    tenant=models.ForeignKey(Tenant,on_delete=models.CASCADE,related_name='users')
    role=models.CharField(max_length=20,choices=Role.choices,default=Role.member)


    def __str__(self):
        return f'{self.username} {self.tenant}'

# Create your models here.
```

File: users\permissions.py

```
from rest_framework.permissions import BasePermission


class IsInSameTenant(BasePermission):
    """
    Object?level permission ? ensure the requesting user and the object
    belong to the same tenant.
    """
    def has_object_permission(self, request, view, obj):
        return (
            request.user.is_authenticated
            and hasattr(obj, 'tenant')
            and request.user.tenant == obj.tenant
        )


class IsTenantAdmin(BasePermission):
    """
    Allows access only to tenant administrators (role == 'admin').
    """
    def has_permission(self, request, view):
        return (
            request.user.is_authenticated
            and request.user.role == 'admin'
        )
```

File: users\serializers.py

```
from rest_framework import serializers
from . import models

class Userregistrationserializer(serializers.ModelSerializer):

    class Meta():
        model=models.User
        fields='__all__'
```

File: users\token.py

```
import logging
from typing import Any
from rest_framework import status
from rest_framework.request import Request
from rest_framework.response import Response
from rest_framework_simplejwt.serializers import TokenObtainPairSerializer
from rest_framework_simplejwt.views import TokenObtainPairView, TokenRefreshView

logger = logging.getLogger(__name__)


class CustomTokenObtainPairSerializer(TokenObtainPairSerializer):
    @classmethod
    def get_token(cls, user: Any) -> Any:
        token = super().get_token(user)
        token['tenant_slug'] = user.tenant.slug
        token['role'] = user.role
        return token


class LoggingTokenObtainPairView(TokenObtainPairView):
    serializer_class = CustomTokenObtainPairSerializer

    def post(self, request: Request, *args: Any, **kwargs: Any) -> Response:
        username = request.data.get('username', 'unknown')
        try:
            response = super().post(request, *args, **kwargs)
            if response.status_code == status.HTTP_200_OK:
                logger.info("Login successful for user: %s", username)
            return response
        except Exception as exc:
            logger.warning("Failed login attempt for user '%s': %s", username, exc)
            raise


class LoggingTokenRefreshView(TokenRefreshView):
    def post(self, request: Request, *args: Any, **kwargs: Any) -> Response:
        try:
            response = super().post(request, *args, **kwargs)
            if response.status_code == status.HTTP_200_OK:
                logger.info("Token refreshed successfully.")
            return response
        except Exception as exc:
            logger.warning("Token refresh failed: %s", exc)
            raise
```

File: users\urls.py

```
from django.urls import path
from .token import LoggingTokenObtainPairView, LoggingTokenRefreshView

urlpatterns = [
    path('token/', LoggingTokenObtainPairView.as_view(), name='token_obtain_pair'),
    path('token/refresh/', LoggingTokenRefreshView.as_view(), name='token_refresh'),
]
```

File: users\views.py

```
from django.shortcuts import render
from rest_framework_simplejwt  .  views import  TokenObtainPairView
from ..users import token
from rest_framework import generics, status
from rest_framework.response import Response
from .serializers import Userregistrationserializer
from .models import User



class Userregistrationview(generics.GenericAPIView):
    queryset=User.objects.all()
    serializer_class=Userregistrationserializer



# Create your views here.
```

File: users\tests__init__.py

File: users\tests\tests.py

```
from __future__ import annotations
import jwt
from django.contrib.auth import get_user_model
from django.test import TestCase, override_settings
from django.urls import path
from rest_framework import status
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response
from rest_framework.test import APIClient, APITestCase
from rest_framework.views import APIView
from tenants.models import Tenant
from users.models import User
from users.permissions import IsInSameTenant

User = get_user_model()


class AuthenticationTests(APITestCase):
    """Tests for JWT token obtain, refresh, and custom claims."""

    def setUp(self) -> None:
        self.tenant = Tenant.objects.create(
            name="TestCo", slug="testco", api_key="key-12345"
        )
        self.admin_user = User.objects.create_user(
            username="admin",
            email="admin@testco.com",
            password="AdminPass1!",
            tenant=self.tenant,
            role="admin",
        )
        self.member_user = User.objects.create_user(
            username="member",
            email="member@testco.com",
            password="MemberPass1!",
            tenant=self.tenant,
            role="member",
        )
        self.client = APIClient()

    def test_token_contains_custom_claims(self) -> None:
        response = self.client.post(
            "/api/v1/token/",
            {"username": "admin", "password": "AdminPass1!"},
            format="json",
        )
        self.assertEqual(response.status_code, status.HTTP_200_OK)
        access_token = response.data["access"]
        payload = jwt.decode(access_token, options={"verify_signature": False})
        self.assertIn("tenant_slug", payload)
        self.assertEqual(payload["tenant_slug"], "testco")
        self.assertIn("role", payload)
        self.assertEqual(payload["role"], "admin")

    def test_token_obtain_and_refresh(self) -> None:
        response = self.client.post(
            "/api/v1/token/",
            {"username": "admin", "password": "AdminPass1!"},
            format="json",
        )
        self.assertEqual(response.status_code, status.HTTP_200_OK)
        refresh_token = response.data["refresh"]

        refresh_response = self.client.post(
            "/api/v1/token/refresh/",
```

```
            {"refresh": refresh_token},
            format="json",
        )
        self.assertEqual(refresh_response.status_code, status.HTTP_200_OK)
        self.assertIn("access", refresh_response.data)

    def test_invalid_credentials(self) -> None:
        response = self.client.post(
            "/api/v1/token/",
            {"username": "admin", "password": "wrong"},
            format="json",
        )
        self.assertEqual(response.status_code, status.HTTP_401_UNAUTHORIZED)

    def test_refresh_with_invalid_token(self) -> None:
        response = self.client.post(
            "/api/v1/token/refresh/",
            {"refresh": "thisisnotarealtoken"},
            format="json",
        )
        self.assertEqual(response.status_code, status.HTTP_401_UNAUTHORIZED)


class TenantIsolationTests(APITestCase):
    """Ensure users can only access objects belonging to their own tenant."""

    @override_settings(ROOT_URLCONF="users.tests.urls")
    def setUp(self) -> None:
        self.tenant_a = Tenant.objects.create(
            name="Tenant A", slug="tenant-a", api_key="key-a"
        )
        self.tenant_b = Tenant.objects.create(
            name="Tenant B", slug="tenant-b", api_key="key-b"
        )
        self.user_a = User.objects.create_user(
            username="usera",
            email="user@a.com",
            password="Pass123!",
            tenant=self.tenant_a,
            role="member",
        )
        self.user_b = User.objects.create_user(
            username="userb",
            email="user@b.com",
            password="Pass123!",
            tenant=self.tenant_b,
            role="member",
        )
        self.client = APIClient()

    def _obtain_token(self, username: str, password: str) -> str:
        resp = self.client.post(
            "/api/v1/token/",
            {"username": username, "password": password},
            format="json",
        )
        return resp.data["access"]

    def test_access_own_tenant_object(self) -> None:
        token = self._obtain_token("usera", "Pass123!")
        self.client.credentials(HTTP_AUTHORIZATION=f"Bearer {token}")
        response = self.client.get("/api/dummy/tenant-a/")
        self.assertEqual(response.status_code, status.HTTP_200_OK)

    def test_cannot_access_other_tenant_object(self) -> None:
        token = self._obtain_token("usera", "Pass123!")
        self.client.credentials(HTTP_AUTHORIZATION=f"Bearer {token}")
```

```
response = self.client.get("/api/dummy/tenant-b/")
self.assertEqual(response.status_code, status.HTTP_403_FORBIDDEN)
```

File: users\tests\urls.py

```
from django.urls import path

from .views import DummyDetailView

urlpatterns = [
    path(
        "api/dummy/<slug:tenant_slug>/",
        DummyDetailView.as_view(),
        name="dummy-detail",
    ),
]
```

File: users\tests\views.py

```
from rest_framework import status
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response
from rest_framework.views import APIView

from tenants.models import Tenant
from users.permissions import IsInSameTenant


class DummyModel:
    """A simple object that carries a tenant for permission testing."""
    def __init__(self, tenant: Tenant) -> None:
        self.tenant = tenant


class DummyDetailView(APIView):
    """Dummy view that checks object-level permissions and returns 200/403."""
    permission_classes = [IsAuthenticated, IsInSameTenant]

    def get(self, request, tenant_slug: str) -> Response:
        try:
            tenant = Tenant.objects.get(slug=tenant_slug)
        except Tenant.DoesNotExist:
            return Response(status=status.HTTP_404_NOT_FOUND)

        obj = DummyModel(tenant)
        self.check_object_permissions(request, obj)
        return Response({"detail": "allowed"})
```